

Proyecto guiado: TaskFlow (React + Spring Boot + PostgreSQL + Docker)

Oct 3, 2026 · @Fer

Qué vamos a construir

Vas a construir **TaskFlow**, un gestor de tareas con frontend React, API REST en Spring Boot, PostgreSQL, envío de correo y todo corriendo en Docker. Es pequeño a propósito: tiene las mismas piezas que un sistema real, pero cabe en un fin de semana.

Lo que hace la app: crear, listar, completar y borrar tareas. Al crear una tarea, el backend envía un correo de notificación.

Stack:

Capa	Tecnología	Por qué
Frontend	React + Vite	Lo más común en ofertas; Vite arranca rápido
Backend	Java 17 + Spring Boot 3	Tu stack principal
Persistencia	Spring Data JPA + Flyway	ORM + migraciones versionadas, como en producción
Base de datos	PostgreSQL 16	Estándar en backend moderno
Integración	Spring Mail + Mailpit	Simula un servicio externo de correo sin cuentas reales
Tests	JUnit 5, Mockito, MockMvc, Testcontainers	Los tres niveles que preguntan en entrevista
Infraestructura	Docker + Docker Compose	Levantar todo con un comando

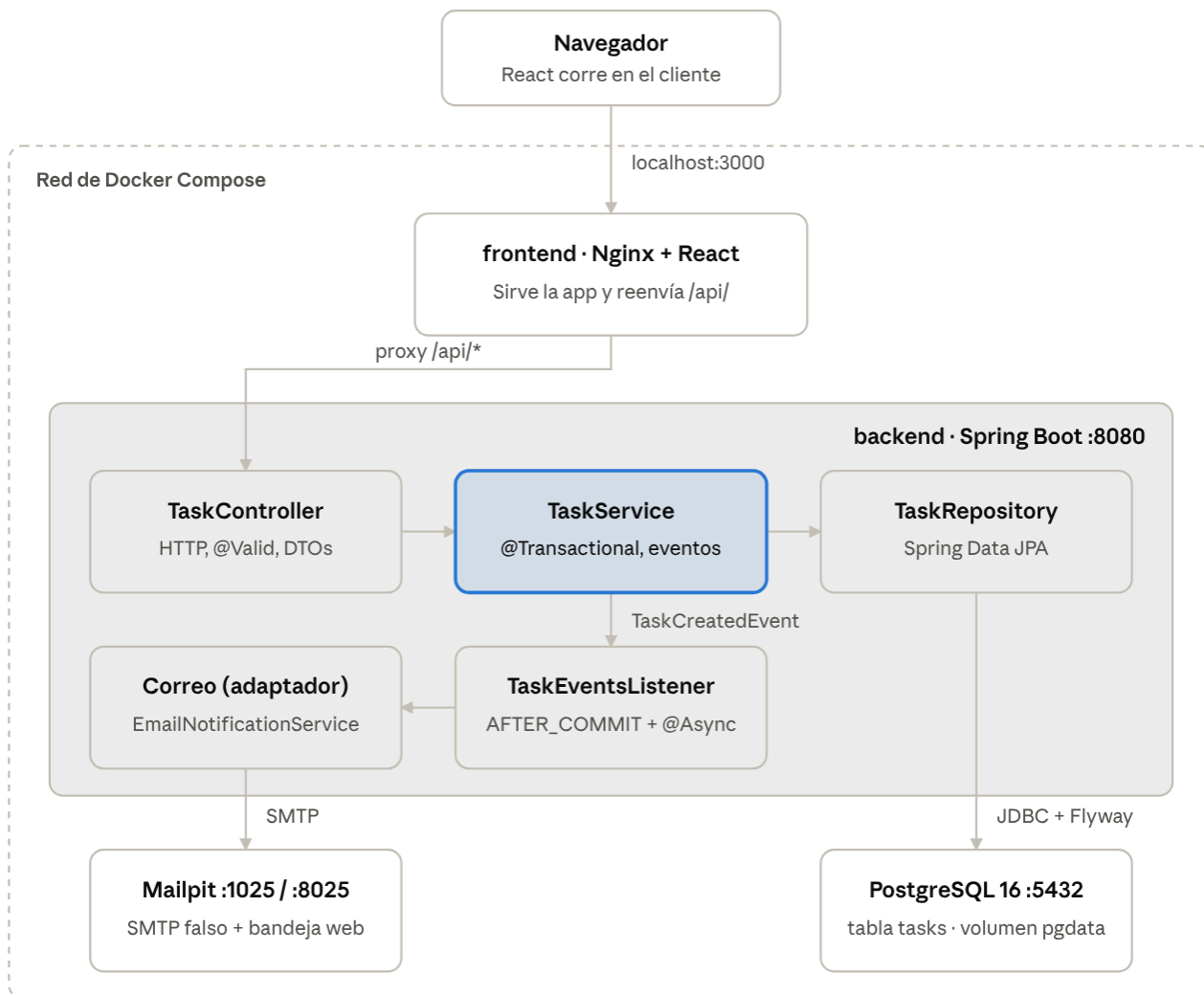
Cómo usar esta clase:

- Cada paso tiene: **Qué hacemos** → **Por qué** → **Código** → **Comprueba**. No pases al siguiente paso sin que el "Comprueba" funcione.
- El código está como referencia. Para entrenar la entrevista: lee el bloque, ciérralo y **escríbelo tú a mano**; luego compara.
- Al final de cada paso hay una pregunta de entrevista. Respóndela en voz alta.
- Tiempo estimado: 8-10 horas en total, en ciclos de 1-2 pasos por sesión.

Requisitos instalados: JDK 17, Maven (o el wrapper `mvnw`), Node.js 20+, Docker Desktop, Git, un editor (IntelliJ con la IA desactivada o VS Code), y Postman o `curl`.

Arquitectura

Una petición viaja de React a Postgres; el correo sale tras el commit



Arquitectura de TaskFlow · 4 contenedores en una red de Compose

La petición entra por Nginx y recorre controller → servicio → repositorio hasta PostgreSQL. El correo no se envía en la petición: el servicio publica un evento y un listener lo manda en otro hilo, solo si la transacción se confirmó. Vuelve a este diagrama al terminar cada paso para ubicar qué pieza acabas de construir.

Paso 1 — Preparar el entorno y la estructura

Qué hacemos: crear un repositorio "monorepo" con dos aplicaciones (backend y frontend) y la infraestructura en la raíz.

Por qué: en empresas reales cada pieza vive separada y se despliega por su cuenta, pero en un proyecto pequeño un solo repo facilita versionar todo junto. Así también lo explicas fácil en entrevista.

1.1 Estructura final del repositorio

```
taskflow/
├── backend/           ← API Spring Boot
├── frontend/         ← App React (Vite)
├── docker-compose.yml ← Orquesta Postgres, Mailpit, API y frontend
├── .env              ← Variables (contraseñas) – NO se sube a Git
└── .gitignore
```

1.2 Crear el backend

En un trabajo real se usa Spring Initializr; no es "vibe coding", es el estándar. Lo que cuenta es que entiendas cada dependencia.

```
mkdir taskflow && cd taskflow && git init

curl https://start.spring.io/starter.zip \
  -d type=maven-project -d javaVersion=17 \
  -d groupId=com.taskflow -d artifactId=backend -d name=taskflow \
  -d packageName=com.taskflow \
  -d dependencies=web,data-jpa,postgresql,flyway,validation,mail,actuator,testcontainers \
  -o backend.zip

unzip backend.zip -d backend && rm backend.zip
```

Qué aporta cada dependencia (apréndetelo, lo preguntan):

Starter	Para qué sirve
web	Spring MVC + Tomcat embebido + Jackson (JSON)
data-jpa	Hibernate + repositorios Spring Data
postgresql	Driver JDBC de PostgreSQL
flyway	Migraciones SQL versionadas
validation	Bean Validation (@NotBlank , @Size ...)
mail	JavaMailSender para enviar correo por SMTP

Starter	Para qué sirve
actuator	Endpoints de salud y métricas (/actuator/health)
testcontainers	Bases de datos reales en Docker para los tests

1.3 Estructura del backend: por funcionalidad, con capas dentro

```

backend/src/main/java/com/taskflow/
├─ TaskflowApplication.java      ← punto de entrada
├─ task/                        ← todo lo de "tareas" junto
│   ├─ Task.java                ← entidad JPA (tabla tasks)
│   ├─ TaskRepository.java      ← acceso a datos
│   ├─ TaskService.java         ← reglas de negocio
│   ├─ TaskController.java      ← endpoints HTTP
│   ├─ TaskNotFoundException.java
│   └─ dto/
│       ├─ TaskRequest.java     ← lo que entra
│       └─ TaskResponse.java    ← lo que sale
├─ notification/                ← integración de correo
│   ├─ NotificationService.java ← interfaz (puerto)
│   └─ EmailNotificationService.java ← implementación (adaptador)
├─ common/
│   └─ GlobalExceptionHandler.java
└─ config/
    └─ WebConfig.java           ← CORS

backend/src/main/resources/
├─ application.yml
└─ db/migration/
    └─ V1__create_tasks.sql

```

Por qué "por funcionalidad" y no carpetas `controllers/`, `services/` : cuando la app crece, todo lo de una funcionalidad queda junto y es fácil convertirlo en módulo o microservicio. Las capas siguen existiendo dentro.

1.4 .gitignore y .env en la raíz

```
# .gitignore
.env
backend/target/
frontend/node_modules/
frontend/dist/
.idea/
```

```
# .env (valores locales, nunca a Git)
POSTGRES_DB=taskflow
POSTGRES_USER=taskflow
POSTGRES_PASSWORD=taskflow_dev
```

Comprueba: `cd backend && ./mvnw -v` muestra Maven y Java 17.

Pregunta de entrevista: ¿Por qué las contraseñas van en variables de entorno y no en `application.yml`? (Respuesta: para no exponer secretos en el repositorio y poder cambiar valores por ambiente sin recompilar — principio 12-Factor App.)

Paso 2 — Base de datos: PostgreSQL en Docker + Flyway

Qué hacemos: levantar PostgreSQL en un contenedor, conectar Spring Boot a él y crear la tabla con una migración.

Por qué: nadie instala Postgres "a mano" en un equipo moderno; cada dev levanta la misma versión con Docker. Y el esquema no lo crea Hibernate: lo controlan migraciones versionadas que se revisan en code review, igual que el código.

2.1 docker-compose.yml (primera versión, solo base de datos)

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

Línea por línea lo importante:

- `image: postgres:16-alpine` → versión fija; nunca `latest` (rompe builds sin avisar).
- `${POSTGRES_DB}` → Compose lee el archivo `.env` de la raíz automáticamente.
- `ports: "5432:5432"` → puerto de tu máquina : puerto del contenedor. Así tu backend local llega a la base.
- `volumes: pgdata` → los datos sobreviven aunque borres el contenedor.
- `healthcheck` → Docker sabe cuándo Postgres ya acepta conexiones (lo usaremos en el Paso 7).

```
docker compose up -d postgres
docker compose ps          # debe decir "healthy"
```

2.2 Conectar Spring Boot: application.yml

Borra `application.properties` y crea `src/main/resources/application.yml`:

```

spring:
  application:
    name: taskflow
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/taskflow}
    username: ${DB_USER:taskflow}
    password: ${DB_PASSWORD:taskflow_dev}
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
    show-sql: true

server:
  port: 8080

```

- `${DB_URL:valor}` → usa la variable de entorno si existe; si no, el valor por defecto para desarrollo local. En Docker cambiaremos la variable, no el código.
- `ddl-auto: validate` → Hibernate **solo verifica** que las entidades coincidan con las tablas. Si no coinciden, la app no arranca: error temprano en lugar de datos corruptos.
- `open-in-view: false` → cierra la sesión de base de datos al terminar el servicio. Evita consultas escondidas desde el controller (y te obliga a usar DTOs).
- `show-sql: true` → solo en desarrollo, para ver qué SQL genera Hibernate.

2.3 Primera migración

Crea `src/main/resources/db/migration/V1__create_tasks.sql` (dos guiones bajos después de `V1`):

```

CREATE TABLE tasks (
  id          BIGSERIAL PRIMARY KEY,
  title       VARCHAR(120) NOT NULL,
  description  VARCHAR(500),
  completed   BOOLEAN      NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
  version     BIGINT       NOT NULL DEFAULT 0
);

CREATE INDEX idx_tasks_completed ON tasks (completed);

```

- `BIGSERIAL` → ID autoincremental (lo genera la base).
- `version` → para bloqueo optimista (lo explicamos en la entidad).

- Índice en `completed` → filtraremos tareas pendientes/completadas. Un índice acelera lecturas pero cuesta en escrituras; ponlos donde consultas.
- **Regla de oro:** una migración aplicada nunca se edita. Si necesitas cambiar algo, creas `V2__...sql`. Flyway guarda un checksum de cada archivo en la tabla `flyway_schema_history`.

Comprueba: aún no hay entidades, así que la app arranca sin endpoints; solo queremos ver la migración. Corre `./mvnw spring-boot:run` y busca en el log `Successfully applied 1 migration`. Luego entra a la base:

```
docker compose exec postgres psql -U taskflow -d taskflow -c "\d tasks"
```

Pregunta de entrevista: ¿Por qué no usar `ddl-auto: update` en producción? (No borra columnas, no maneja renombres, no se puede revisar ni revertir, y cada instancia podría intentar alterar el esquema a la vez.)

Paso 3 — Backend capa por capa

Qué hacemos: construir el flujo completo de una petición: Controller → Service → Repository → Base de datos, con DTOs y manejo de errores.

Por qué así: cada capa tiene una sola responsabilidad (la S de SOLID). El controller solo traduce HTTP; el servicio tiene las reglas de negocio y las transacciones; el repositorio solo habla con la base. Si mañana cambias REST por GraphQL o Postgres por otra base, cambias una capa, no todas.

Orden recomendado (de adentro hacia afuera): entidad → repositorio → DTOs → excepción → servicio → controller → manejador de errores.

3.1 Entidad `Task` — el modelo que mapea la tabla

```
package com.taskflow.task;

import jakarta.persistence.*;
import java.time.Instant;

@Entity
@Table(name = "tasks")
public class Task {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

@Column(nullable = false, length = 120)
private String title;

@Column(length = 500)
private String description;

@Column(nullable = false)
private boolean completed;

@Column(name = "created_at", nullable = false, updatable = false)
private Instant createdAt;

@Version
private Long version;

protected Task() { } // JPA lo necesita; nadie más lo usa

public Task(String title, String description) {
    this.title = title;
    this.description = description;
    this.completed = false;
    this.createdAt = Instant.now();
}

public void update(String title, String description) {
    this.title = title;
    this.description = description;
}

public void complete() {
    this.completed = true;
}

public Long getId() { return id; }
public String getTitle() { return title; }
public String getDescription() { return description; }
public boolean isCompleted() { return completed; }
public Instant getCreatedAt() { return createdAt; }
}

```

- **Sin setters públicos:** los cambios pasan por métodos con intención (`complete()` , `update()`). Es encapsulamiento real; en entrevista suma puntos frente a una entidad llena de setters.
- **IDENTITY** → el ID lo genera Postgres (`BIGSERIAL`).

- `@Version` → **bloqueo optimista**: si dos usuarios editan la misma tarea, el segundo recibe un error en vez de pisar el cambio del primero.
- Constructor `protected` vacío → Hibernate crea objetos por reflexión.

3.2 Repositorio — acceso a datos sin escribir SQL

```
package com.taskflow.task;

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface TaskRepository extends JpaRepository<Task, Long> {

    List<Task> findByCompletedOrderByCreatedAtDesc(boolean completed);
}
```

- Es solo una interfaz: Spring Data genera la implementación al arrancar (con un proxy).
- `JpaRepository` ya trae `save`, `findById`, `findAll`, `deleteById`, `existsById`, paginación y ordenamiento.
- `findByCompletedOrderByCreatedAtDesc` → **query derivada**: Spring traduce el nombre a `SELECT ... WHERE completed = ? ORDER BY created_at DESC`.

3.3 DTOs — el contrato de la API

Carpeta `task/dto/`. Usamos `record` (inmutables, sin boilerplate).

```
package com.taskflow.task.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record TaskRequest(
    @NotBlank(message = "El título es obligatorio")
    @Size(max = 120, message = "Máximo 120 caracteres")
    String title,

    @Size(max = 500, message = "Máximo 500 caracteres")
    String description
) { }
```

```
package com.taskflow.task.dto;

import com.taskflow.task.Task;
import java.time.Instant;

public record TaskResponse(Long id, String title, String description,
                          boolean completed, Instant createdAt) {

    public static TaskResponse from(Task task) {
        return new TaskResponse(task.getId(), task.getTitle(),
            task.getDescription(),
                task.isCompleted(), task.getCreatedAt());
    }
}
```

Por qué no devolver la entidad directamente: expondrías campos internos (`version`), acoplarías la API a la tabla, y con relaciones lazy tendrías `LazyInitializationException` o JSON infinito. El DTO es el contrato público; la entidad es un detalle interno.

3.4 Excepción de negocio

```
package com.taskflow.task;

public class TaskNotFoundException extends RuntimeException {
    public TaskNotFoundException(Long id) {
        super("La tarea " + id + " no existe");
    }
}
```

Es `RuntimeException` (unchecked) para que no ensucie las firmas y para que `@Transactional` haga rollback automático.

3.5 Servicio — reglas de negocio y transacciones

```
package com.taskflow.task;

import com.taskflow.task.dto.TaskRequest;
import com.taskflow.task.dto.TaskResponse;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
@Transactional(readOnly = true)
```

```

public class TaskService {

    private final TaskRepository repository;

    public TaskService(TaskRepository repository) {    // inyección por
constructor
        this.repository = repository;
    }

    public List<TaskResponse> findAll(Boolean completed) {
        List<Task> tasks = (completed == null)
            ? repository.findAll(Sort.by(Sort.Direction.DESC,
"createdAt"))
            : repository.findByCompletedOrderByCreatedAtDesc(completed);
        return tasks.stream().map(TaskResponse::from).toList();
    }

    public TaskResponse findById(Long id) {
        return TaskResponse.from(getTask(id));
    }

    @Transactional
    public TaskResponse create(TaskRequest request) {
        Task saved = repository.save(new Task(request.title(),
request.description()));
        return TaskResponse.from(saved);
    }

    @Transactional
    public TaskResponse update(Long id, TaskRequest request) {
        Task task = getTask(id);
        task.update(request.title(), request.description());
        return TaskResponse.from(task);           // sin save(): dirty
checking
    }

    @Transactional
    public TaskResponse complete(Long id) {
        Task task = getTask(id);
        task.complete();
        return TaskResponse.from(task);
    }

    @Transactional
    public void delete(Long id) {
        if (!repository.existsById(id)) {

```

```

        throw new TaskNotFoundException(id);
    }
    repository.deleteById(id);
}

private Task getTask(Long id) {
    return repository.findById(id).orElseThrow(() -> new
TaskNotFoundException(id));
}
}

```

- `@Transactional(readOnly = true)` en la clase → todas las lecturas son de solo lectura; los métodos que escriben lo sobrescriben con `@Transactional`.
- **Dirty checking:** en `update` no llamamos `save()`. La entidad está "managed" dentro de la transacción; al hacer commit, Hibernate detecta el cambio y ejecuta el `UPDATE` solo. Pregunta clásica.
- El servicio devuelve DTOs, así la transacción termina antes de serializar a JSON.

3.6 Controller — la puerta HTTP

```

package com.taskflow.task;

import com.taskflow.task.dto.TaskRequest;
import com.taskflow.task.dto.TaskResponse;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    private final TaskService service;

    public TaskController(TaskService service) {
        this.service = service;
    }

    @GetMapping
    public List<TaskResponse> findAll(@RequestParam(required = false) Boolean
completed) {
        return service.findAll(completed);
    }
}

```

```

    }

    @GetMapping("/{id}")
    public TaskResponse findById(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    public ResponseEntity<TaskResponse> create(@Valid @RequestBody
TaskRequest request) {
        TaskResponse created = service.create(request);
        return ResponseEntity.created(URI.create("/api/tasks/" +
created.id())).body(created);
    }

    @PutMapping("/{id}")
    public TaskResponse update(@PathVariable Long id, @Valid @RequestBody
TaskRequest request) {
        return service.update(id, request);
    }

    @PatchMapping("/{id}/complete")
    public TaskResponse complete(@PathVariable Long id) {
        return service.complete(id);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.delete(id);
    }
}

```

Endpoint	Status de éxito	Error posible
GET /api/tasks?completed=false	200	—
GET /api/tasks/{id}	200	404
POST /api/tasks	201 + header Location	400
PUT /api/tasks/{id}	200	400, 404, 409
PATCH /api/tasks/{id}/complete	200	404
DELETE /api/tasks/{id}	204 sin cuerpo	404

- `@Valid` → activa las validaciones del record; si fallan, Spring lanza `MethodArgumentNotValidException` antes de entrar al método.
- `PATCH` para completar → cambio parcial de estado; `PUT` reemplaza el recurso completo.

3.7 Manejo global de errores con `ProblemDetail`

```
package com.taskflow.common;

import com.taskflow.task.TaskNotFoundException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.orm.ObjectOptimisticLockingFailureException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(TaskNotFoundException.class)
    public ProblemDetail handleNotFound(TaskNotFoundException ex) {
        ProblemDetail problem =
ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Recurso no encontrado");
        return problem;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex)
    {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(error -> errors.put(error.getField(),
error.getDefaultMessage()));

        ProblemDetail problem =
ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        problem.setTitle("Datos inválidos");
        problem.setProperty("errors", errors);
        return problem;
    }
}
```

```

    @ExceptionHandler(ObjectOptimisticLockingFailureException.class)
    public ProblemDetail
    handleConflict(ObjectOptimisticLockingFailureException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT,
            "La tarea fue modificada por otro usuario. Recarga e intenta
de nuevo.");
    }
}

```

`ProblemDetail` es el formato estándar RFC 9457 para errores HTTP (Spring 6 / Boot 3). El frontend siempre recibe la misma forma: `title`, `status`, `detail` y, en validación, `errors` por campo.

Comprueba (con Postgres arriba y `./mvnw spring-boot:run`):

```

# Crear → 201
curl -i -X POST localhost:8080/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"Repasar Spring","description":"Día 3 de la guía"}'

# Listar → 200
curl localhost:8080/api/tasks

# Validación → 400 con errors.title
curl -i -X POST localhost:8080/api/tasks -H "Content-Type: application/json"
-d '{"title":""}'

# Completar → 200
curl -i -X PATCH localhost:8080/api/tasks/1/complete

# No existe → 404
curl -i localhost:8080/api/tasks/999

```

Revisa en la consola el SQL que imprimió Hibernate para cada llamada; en el `PATCH` verás un `UPDATE ... where id=? and version=?` (ese es el bloqueo optimista trabajando).

Pregunta de entrevista: Explica el recorrido de un `POST /api/tasks` desde que llega la petición hasta que se responde 201. (DispatcherServlet → HandlerMapping → deserializa JSON con Jackson → valida con `@Valid` → controller → proxy transaccional abre transacción → servicio → repositorio → `INSERT` → commit → DTO → Jackson serializa → 201 con `Location`.)

Paso 4 — Tests del backend

Qué hacemos: escribir tres tipos de prueba, de la más rápida a la más completa.

Por qué tres niveles (pirámide de testing): muchos tests unitarios (milisegundos, prueban lógica), algunos de capa web (prueban HTTP: status, JSON, validación) y pocos de integración (prueban todo junto con una base real, tardan segundos). En entrevista: "pruebo la lógica aislada con Mockito, el contrato HTTP con `@WebMvcTest` y el flujo real con Testcontainers".

Los tests van en `src/test/java/com/taskflow/task/`. Usamos JUnit 5 + AssertJ + Mockito (ya vienen en `spring-boot-starter-test`).

4.1 Test unitario del servicio (sin Spring)

```
package com.taskflow.task;

import com.taskflow.task.dto.TaskRequest;
import com.taskflow.task.dto.TaskResponse;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class TaskServiceTest {

    @Mock
    TaskRepository repository;

    @InjectMocks
    TaskService service;

    @Test
    void create_guardaLaTareaYDevuelveDto() {
        when(repository.save(any(Task.class))).thenReturn(inv ->
            inv.getArgument(0));

        TaskResponse response = service.create(new TaskRequest("Estudiar",
            "JPA"));

        assertThat(response.title()).isEqualTo("Estudiar");
        assertThat(response.completed()).isFalse();
    }
}
```

```

        verify(repository).save(any(Task.class));
    }

    @Test
    void findById_lanzaExcepcionSiNoExiste() {
        when(repository.findById(99L)).thenReturn(Optional.empty());

        assertThatThrownBy(() -> service.findById(99L))
            .assertInstanceOf(TaskNotFoundException.class)
            .hasMessageContaining("99");
    }

    @Test
    void complete_marcaLaTareaComoCompletada() {
        when(repository.findById(1L)).thenReturn(Optional.of(new
Task("Estudiar", null)));

        TaskResponse response = service.complete(1L);

        assertThat(response.completed()).isTrue();
    }
}

```

- Estructura **Arrange / Act / Assert** (preparar, ejecutar, verificar), separada por líneas en blanco.
- `@Mock` crea un repositorio falso; `@InjectMocks` crea el servicio real inyectándole el mock por constructor.
- `thenAnswer(inv -> inv.getArgument(0))` → el mock devuelve lo mismo que recibe, como haría `save`.
- Nombres de test que describen el comportamiento: si falla, el nombre ya te dice qué se rompió.

4.2 Test de la capa web con `MockMvc`

```

package com.taskflow.task;

import com.taskflow.task.dto.TaskResponse;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;
import java.time.Instant;

```

```

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(TaskController.class)
class TaskControllerTest {

    @Autowired
    MockMvc mockMvc;

    @MockitoBean          // en Spring Boot anterior a 3.4: @MockBean
    TaskService service;

    @Test
    void post_valido_responde201ConLocation() throws Exception {
        when(service.create(any()))
            .thenReturn(new TaskResponse(1L, "Estudiar", null, false,
Instant.now()));

        mockMvc.perform(post("/api/tasks")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                {"title": "Estudiar"}
            """))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location", "/api/tasks/1"))
            .andExpect(jsonPath("$.title").value("Estudiar"));
    }

    @Test
    void post_sinTitulo_responde400() throws Exception {
        mockMvc.perform(post("/api/tasks")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"title\": \"\"}"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors.title").exists());

        verifyNoInteractions(service);
    }

    @Test
    void get_inexistente_responde404() throws Exception {

```

```

        when(service.findById(99L)).thenThrow(new
TaskNotFoundException(99L));

        mockMvc.perform(get("/api/tasks/99"))
                .andExpect(status().isNotFound());
    }
}

```

- `@WebMvcTest` levanta **solo** la capa web: controllers, `@RestControllerAdvice`, Jackson y validación. No hay base de datos.
- El servicio es un mock: aquí probamos el **contrato HTTP**, no la lógica.
- `verifyNoInteractions(service)` demuestra que la validación frenó la petición antes de llegar al servicio.

4.3 Test de integración con Testcontainers

```

package com.taskflow.task;

import com.taskflow.task.dto.TaskRequest;
import com.taskflow.task.dto.TaskResponse;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.testcontainers.service.connection.ServiceConnection;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Testcontainers
class TaskIntegrationTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16-alpine");

    @Autowired
    TestRestTemplate rest;

```

```

@Test
void crearYConsultarUnaTarea() {
    ResponseEntity<TaskResponse> created = rest.postForEntity(
        "/api/tasks", new TaskRequest("Integración", "Postgres
real"), TaskResponse.class);

    assertThat(created.getStatusCode()).isEqualTo(HttpStatus.CREATED);

    Long id = created.getBody().id();
    TaskResponse found = rest.getForObject("/api/tasks/" + id,
TaskResponse.class);

    assertThat(found.title()).isEqualTo("Integración");
    assertThat(found.completed()).isFalse();
}
}

```

- Testcontainers arranca un Postgres **real y desechable** en Docker para el test; Flyway corre la migración igual que en producción.
- `@ServiceConnection` (Boot 3.1+) conecta el datasource al contenedor sin configurar URL a mano.
- `RANDOM_PORT` levanta el servidor real; `TestRestTemplate` hace peticiones HTTP de verdad.
- **Por qué no H2:** H2 no se comporta igual que Postgres (tipos, SQL, índices). Con Testcontainers pruebas contra lo mismo que corre en producción.

Comprueba: con Docker abierto, `./mvnw test` → todos en verde. El test de integración tarda más la primera vez porque descarga la imagen.

Pregunta de entrevista: ¿Diferencia entre `@Mock` y `@MockitoBean` / `@MockBean` ? (`@Mock` es Mockito puro, sin Spring; `@MockitoBean` reemplaza un bean dentro del contexto de Spring del test.)

Paso 5 — Frontend React y conexión al backend

Qué hacemos: crear una app React mínima que liste, cree, complete y borre tareas llamando a la API.

Por qué así: el foco es la **conexión**, no el diseño. Separamos las llamadas HTTP en un archivo propio (`api/tasks.js`) para que los componentes no sepan de URLs ni de `fetch`: es la misma idea de capas del backend, aplicada al frontend.

5.1 Crear el proyecto

```
cd taskflow
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev          # abre http://localhost:5173
```

Borra el contenido de ejemplo de `App.jsx` y `App.css`. Estructura que vamos a usar:

```
frontend/src/
├── api/
│   └── tasks.js    ← todas las llamadas HTTP
├── App.jsx         ← pantalla principal
└── main.jsx        ← punto de entrada (ya existe)
```

5.2 El problema de CORS (y cómo resolverlo)

Tu frontend corre en `http://localhost:5173` y el backend en `http://localhost:8080`. Para el navegador son **orígenes distintos** (cambia el puerto), así que bloquea la respuesta a menos que el backend diga explícitamente "acepto peticiones de ese origen". Eso es **CORS**: una protección del navegador, no del servidor (por eso `curl` sí funcionaba).

En el backend, crea `config/WebConfig.java`:

```
package com.taskflow.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Value("${app.cors.allowed-origins:http://localhost:5173}")
    private String[] allowedOrigins;

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins(allowedOrigins)
            .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE");
    }
}
```

- El origen permitido sale de configuración, no está fijo en código: en producción será tu dominio real.
- Nunca uses `allowedOrigins("*")` en una API con datos de usuarios.
- Para métodos como `PATCH` o `DELETE` con JSON, el navegador primero manda un `OPTIONS (preflight)` para preguntar si está permitido. Spring lo responde con esta configuración.

5.3 Capa de acceso a la API: `src/api/tasks.js`

```
const API_URL = import.meta.env.VITE_API_URL ?? 'http://localhost:8080';

async function request(path, options = {}) {
  const response = await fetch(`${API_URL}${path}`, {
    headers: { 'Content-Type': 'application/json' },
    ...options,
  });

  if (!response.ok) {
    const problem = await response.json().catch(() => ({}));
    const fieldErrors = problem.errors ?
Object.values(problem.errors).join(', ') : null;
    throw new Error(fieldErrors ?? problem.detail ?? problem.title ?? `Error
${response.status}`);
  }

  return response.status === 204 ? null : response.json();
}

export const getTasks = () => request('/api/tasks');

export const createTask = (task) =>
  request('/api/tasks', { method: 'POST', body: JSON.stringify(task) });

export const completeTask = (id) =>
  request(`/api/tasks/${id}/complete`, { method: 'PATCH' });

export const deleteTask = (id) =>
  request(`/api/tasks/${id}`, { method: 'DELETE' });
```

- `import.meta.env.VITE_API_URL` → variable de entorno de Vite (deben empezar con `VITE_`). Así la URL cambia por ambiente sin tocar código, igual que en el backend.
- `fetch` **no** lanza error con 4xx/5xx; solo con fallos de red. Por eso revisamos `response.ok` a mano. Pregunta frecuente.

- Leemos el `ProblemDetail` del backend para mostrar el mensaje real (`detail` o los `errors` de validación).
- `204` no tiene cuerpo: si intentas `response.json()` falla; por eso devolvemos `null`.

5.4 Pantalla: `src/App.jsx`

```
import { useEffect, useState } from 'react';
import { getTasks, createTask, completeTask, deleteTask } from './api/tasks';

export default function App() {
  const [tasks, setTasks] = useState([]);
  const [title, setTitle] = useState('');
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  async function loadTasks() {
    try {
      setTasks(await getTasks());
      setError(null);
    } catch (e) {
      setError(e.message);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    loadTasks();
  }, []);

  async function run(action) {
    try {
      await action();
      await loadTasks();
    } catch (e) {
      setError(e.message);
    }
  }

  function handleSubmit(event) {
    event.preventDefault();
    run(async () => {
      await createTask({ title, description: '' });
      setTitle('');
    });
  }
}
```

```

    }

    return (
      <main style={{ maxWidth: 560, margin: '40px auto', fontFamily: 'sans-serif' }}>
        <h1>TaskFlow</h1>

        <form onSubmit={handleSubmit}>
          <input value={title} onChange={(e) => setTitle(e.target.value)}
placeholder="Nueva tarea" />
          <button type="submit">Agregar</button>
        </form>

        {error && <p style={{ color: 'crimson' }}>{error}</p>}

        {loading ? (
          <p>Cargando...</p>
        ) : (
          <ul>
            {tasks.map((task) => (
              <li key={task.id}>
                <span style={{ textDecoration: task.completed ? 'line-through'
: 'none' }}>
                  {task.title}
                </span>
                {!task.completed && (
                  <button onClick={() => run(() =>
completeTask(task.id))}>Completar</button>
                )}
                <button onClick={() => run(() =>
deleteTask(task.id))}>Borrar</button>
              </li>
            )}}
          </ul>
        )}
      </main>
    );
  }
}

```

- `useState` guarda el estado de la pantalla; cuando cambia, React vuelve a pintar.
- `useEffect(..., [])` → carga las tareas una vez al montar el componente. En desarrollo, `StrictMode` lo ejecuta dos veces a propósito; no es un bug.
- `key={task.id}` → React identifica cada elemento de la lista; usa siempre un ID estable, nunca el índice.

- `run()` centraliza "ejecuta, recarga, y si falla muestra el error": evitamos repetir `try/catch` en cada botón.

Comprueba:

1. Backend corriendo (`./mvnw spring-boot:run`) y frontend (`npm run dev`).
2. Abre `http://localhost:5173` , crea una tarea, complétala y bórrala.
3. Abre DevTools → pestaña **Network**: verás el `OPTIONS` (preflight) antes del `PATCH` , y las respuestas 201, 200 y 204.
4. Envía el formulario vacío: debe aparecer "El título es obligatorio" (viene del backend).
5. Prueba de CORS: comenta temporalmente `addCorsMappings` , reinicia, y verás el error de CORS en la consola del navegador. Vuelve a activarlo.

Pregunta de entrevista: ¿Qué es CORS y por qué `curl` funcionaba pero el navegador no? (Es una política del navegador para proteger al usuario; el servidor responde igual, pero el navegador bloquea la respuesta si el origen no está permitido.)

Paso 6 — Integración externa: notificación por correo + perfiles

Qué hacemos: cada vez que se crea una tarea, enviar un correo. Usamos **Mailpit**, un servidor SMTP falso que atrapa los correos y los muestra en una web, para simular un proveedor real (SendGrid, SES) sin cuentas ni riesgos.

Por qué es buen material de entrevista: integrar un servicio externo trae problemas reales: ¿qué pasa si el servicio está caído? ¿Mando el correo si la base de datos hizo rollback? ¿Hago esperar al usuario mientras se envía? Aquí resolvemos los tres.

6.1 Agregar Mailpit a `docker-compose.yml`

Dentro de `services:` , debajo de `postgres` :

```
mailpit:
  image: axllent/mailpit:latest
  ports:
    - "1025:1025"    # SMTP: aquí envía el backend
    - "8025:8025"    # Web: aquí ves los correos
```

```
docker compose up -d mailpit
```

Abre `http://localhost:8025` : es tu bandeja de entrada falsa.

6.2 Configuración

Agrega a `application.yml` :

```

spring:
  mail:
    host: ${MAIL_HOST:localhost}
    port: ${MAIL_PORT:1025}

app:
  notifications:
    to: ${NOTIFY_TO:equipo@taskflow.local}
  cors:
    allowed-origins: ${CORS_ORIGINS:http://localhost:5173}

```

(Si ya tienes una clave `spring:` en el archivo, agrega `mail:` dentro de ella; YAML no permite claves repetidas.)

6.3 Puerto y adaptador (arquitectura hexagonal en pequeño)

El dominio define **qué** necesita (una interfaz); la infraestructura decide **cómo** (correo, SMS, Slack). Así puedes cambiar de proveedor sin tocar el servicio de tareas, y en tests reemplazarlo por un mock.

```

package com.taskflow.notification;

import com.taskflow.task.dto.TaskResponse;

public interface NotificationService {
    void taskCreated(TaskResponse task);
}

```

```

package com.taskflow.notification;

import com.taskflow.task.dto.TaskResponse;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.mail.MailException;
import org.springframework.mail.SimpleMailMessage;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.stereotype.Service;

@Service
public class EmailNotificationService implements NotificationService {

    private static final Logger log =
        LoggerFactory.getLogger(EmailNotificationService.class);

```

```

private final JavaMailSender mailSender;
private final String recipient;

public EmailNotificationService(JavaMailSender mailSender,
                                @Value("${app.notifications.to}") String
recipient) {
    this.mailSender = mailSender;
    this.recipient = recipient;
}

@Override
public void taskCreated(TaskResponse task) {
    SimpleMailMessage message = new SimpleMailMessage();
    message.setFrom("no-reply@taskflow.local");
    message.setTo(recipient);
    message.setSubject("Nueva tarea: " + task.title());
    message.setText("Se creó la tarea #" + task.id() + ": " +
task.title());

    try {
        mailSender.send(message);
        log.info("Notificación enviada para la tarea {}", task.id());
    } catch (MailException e) {
        log.warn("No se pudo notificar la tarea {}: {}", task.id(),
e.getMessage());
    }
}
}

```

Decisión de diseño: si el correo falla, lo registramos en el log y seguimos. La tarea ya se guardó; un correo caído no debe romper la funcionalidad principal. (En un sistema más grande: reintentos con backoff o una cola de mensajes.)

6.4 Enviar el correo *después* del commit, sin bloquear al usuario

Si llamáramos al correo dentro de `create()`, habría dos problemas: (1) si la transacción hace rollback, el correo ya salió anunciando una tarea que no existe; (2) el usuario espera a que el SMTP responda. Solución: **evento de dominio + listener transaccional + asíncrono**.

Evento (en el paquete `task`):

```
package com.taskflow.task;

import com.taskflow.task.dto.TaskResponse;

public record TaskCreatedEvent(TaskResponse task) { }
```

En `TaskService`, inyecta el publicador y publica el evento al crear:

```
private final TaskRepository repository;
private final ApplicationEventPublisher events;

public TaskService(TaskRepository repository, ApplicationEventPublisher
events) {
    this.repository = repository;
    this.events = events;
}

@Transactional
public TaskResponse create(TaskRequest request) {
    Task saved = repository.save(new Task(request.title(),
request.description()));
    TaskResponse response = TaskResponse.from(saved);
    events.publishEvent(new TaskCreatedEvent(response));
    return response;
}
```

Listener (en el paquete `notification`):

```
package com.taskflow.notification;

import com.taskflow.task.TaskCreatedEvent;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Component;
import org.springframework.transaction.event.TransactionPhase;
import org.springframework.transaction.event.TransactionalEventListener;

@Component
public class TaskEventsListener {

    private final NotificationService notificationService;

    public TaskEventsListener(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @Async
```

```

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void onTaskCreated(TaskCreatedEvent event) {
        notificationService.taskCreated(event.task());
    }
}

```

Y activa la ejecución asíncrona en la clase principal:

```

@SpringBootApplication
@EnableAsync
public class TaskflowApplication { ... }

```

- `AFTER_COMMIT` → el listener solo corre si la transacción se confirmó. Rollback = no hay correo.
- `@Async` → corre en otro hilo; la respuesta 201 sale sin esperar al SMTP.
- Desacoplamiento: `task` no conoce a `notification`; solo publica un hecho. Mañana puedes agregar otro listener (auditoría, métricas) sin tocar `TaskService`.
- **Actualiza el test unitario:** agrega `@Mock ApplicationEventPublisher events;` en `TaskServiceTest` y, en el test de `create`, `verify(events).publishEvent(any(TaskCreatedEvent.class));`.

6.5 Perfiles: configuración por ambiente

Un **perfil** es un conjunto de configuración que se activa por ambiente. Spring carga `application.yml` siempre y encima `application-<perfil>.yml`.

`application-dev.yml` (tu máquina):

```

spring:
  jpa:
    show-sql: true
logging:
  level:
    com.taskflow: DEBUG

```

`application-prod.yml` (contenedor):

```

spring:
  jpa:
    show-sql: false
logging:
  level:
    root: INFO

```

Quita `show-sql: true` del `application.yml` base. Para correr en local con el perfil dev:

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=dev
```

En Docker activaremos `prod` con la variable `SPRING_PROFILES_ACTIVE=prod`.

Comprueba: crea una tarea desde el frontend → la respuesta llega al instante → abre `http://localhost:8025` y verás el correo "Nueva tarea: ...". Luego detén Mailpit (`docker compose stop mailpit`), crea otra tarea: se guarda bien y en el log del backend aparece el `warn`. Vuelve a levantarlo.

Pregunta de entrevista: ¿Qué pasa si envías un correo dentro de un método `@Transactional` y luego la transacción falla? ¿Cómo lo evitas? (Se envía igual porque el correo no es transaccional; se evita con `@TransactionalEventListener(AFTER_COMMIT)` o, a mayor escala, con el patrón Outbox.)

Paso 7 — Dockerizar todo

Qué hacemos: empaquetar backend y frontend en imágenes Docker y levantar los 4 servicios con un solo comando.

Por qué: "en mi máquina funciona" deja de ser problema. La misma imagen corre en tu laptop, en CI y en un servidor (por ejemplo un droplet). Es el puente entre desarrollo y despliegue.

7.1 Dockerfile del backend (multi-stage)

backend/Dockerfile:

```
# Etapa 1: compilar con Maven
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

# Etapa 2: imagen final, solo con el JRE
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S app && adduser -S app -G app
USER app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- **Multi-stage:** la primera etapa tiene Maven y el JDK (pesada); la final solo el JRE y el `.jar`. Imagen mucho más pequeña y con menos superficie de ataque.
- Copiar `pom.xml` y bajar dependencias **antes** de copiar `src` aprovecha la caché de capas: si solo cambias código, Docker no vuelve a descargar dependencias.
- `USER app` → el proceso no corre como root (buena práctica de seguridad).
- `-DskipTests` → los tests corren en CI, no al construir la imagen (los de integración necesitan Docker).

backend/.dockerignore:

```
target/
.idea/
```

7.2 Dockerfile del frontend + Nginx

En producción React no corre con `npm run dev`: se compila a archivos estáticos (`dist/`) que sirve un servidor web. Usamos Nginx, que además reenvía `/api` al backend.

frontend/nginx.conf:

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        try_files $uri /index.html;
    }

    location /api/ {
        proxy_pass http://backend:8080;
    }
}
```

frontend/Dockerfile:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=""
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

frontend/.dockerignore:

```
node_modules/
dist/
```

- `VITE_API_URL=""` → el frontend llama a rutas relativas (`/api/tasks`). El navegador las manda al mismo Nginx, y Nginx las reenvía al backend. **Mismo origen = sin CORS** en producción.
- `try_files $uri /index.html` → si recargas en una ruta del frontend, Nginx devuelve la app en vez de un 404.
- `npm ci` → instala exactamente lo del `package-lock.json` (reproducible).

7.3 docker-compose.yml final

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      retries: 10
```

```

mailpit:
  image: axllent/mailpit:latest
  ports:
    - "1025:1025"
    - "8025:8025"

backend:
  build: ./backend
  environment:
    SPRING_PROFILES_ACTIVE: prod
    DB_URL: jdbc:postgresql://postgres:5432/${POSTGRES_DB}
    DB_USER: ${POSTGRES_USER}
    DB_PASSWORD: ${POSTGRES_PASSWORD}
    MAIL_HOST: mailpit
    MAIL_PORT: 1025
    CORS_ORIGINS: http://localhost:3000
  ports:
    - "8080:8080"
  depends_on:
    postgres:
      condition: service_healthy
    mailpit:
      condition: service_started
  healthcheck:
    test: ["CMD", "wget", "-qO-", "http://localhost:8080/actuator/health"]
    interval: 10s
    retries: 10

frontend:
  build: ./frontend
  ports:
    - "3000:80"
  depends_on:
    backend:
      condition: service_healthy

volumes:
  pgdata:

```

Lo más importante para entender:

- **Red interna de Compose:** cada servicio se encuentra por su **nombre**. Dentro de Docker, el backend llega a la base con `postgres:5432`, no con `localhost` (dentro de un contenedor, `localhost` es el propio contenedor). Este es el error número uno al dockerizar.

- **Mismo código, otra configuración:** el backend usa las variables `DB_URL` , `MAIL_HOST` ... que definimos en el Paso 2 con valores por defecto. No cambiamos ni una línea de Java.
- `depends_on` con `service_healthy` → el backend no arranca hasta que Postgres acepta conexiones, y el frontend espera a que `/actuator/health` responda.
- Los secretos salen de `.env` . En un servidor real vendrían de un gestor de secretos o de las variables del entorno de despliegue.

7.4 Levantar, revisar, apagar

```
# Detén el backend y el frontend que corrías en local para liberar los
puertos
docker compose up --build -d      # construye y levanta todo
docker compose ps                 # estado y salud de cada servicio
docker compose logs -f backend    # ver logs (busca Flyway y "Started")

docker compose down               # apaga (los datos quedan en el volumen)
docker compose down -v           # apaga y BORRA los datos
```

Comprueba:

URL	Qué debes ver
<code>http://localhost:3000</code>	La app React servida por Nginx
<code>http://localhost:8080/actuator/health</code>	<code>{"status": "UP"}</code>
<code>http://localhost:8025</code>	Los correos de cada tarea creada

Crea tareas, haz `docker compose down` y `up -d` otra vez: las tareas siguen ahí gracias al volumen.

Pregunta de entrevista: ¿Por qué dentro de Docker el backend no puede conectarse a `localhost:5432` ? (Porque cada contenedor tiene su propia red; `localhost` apunta a sí mismo. En la red de Compose se usa el nombre del servicio, que Docker resuelve por DNS interno.)

Cómo contarle en la entrevista y retos extra

Pitch de 60 segundos (practícalo en voz alta):

"Construí TaskFlow, una app full stack con React y una API REST en Spring Boot 3 sobre PostgreSQL. El backend está organizado por funcionalidad con capas controller, servicio y repositorio; uso DTOs con records, validación con Bean Validation y errores estándar con ProblemDetail. El esquema se versiona con Flyway y uso bloqueo optimista con `@Version`. Para la notificación por correo apliqué puerto y adaptador, y la disparo con un evento transaccional después del commit y de forma asíncrona, para no mandar correos de transacciones fallidas ni bloquear al usuario. Lo pruebo en tres niveles: Mockito, MockMvc y Testcontainers con Postgres real. Todo corre con Docker Compose, con Nginx sirviendo el frontend y haciendo proxy a la API."

Decisiones que te pueden pedir defender:

Decisión	Por qué la tomé	Alternativa
Paquetes por funcionalidad	Escala mejor y facilita extraer un módulo	Paquetes por capa
DTOs con <code>record</code>	Desacopla API de base de datos, inmutables	Exponer la entidad (mala práctica)
Flyway + <code>ddl-auto: validate</code>	Esquema versionado y revisable	<code>ddl-auto: update</code>
<code>open-in-view: false</code>	Evita consultas ocultas en la vista	Default de Spring (true)
Evento <code>AFTER_COMMIT</code> + <code>@Async</code>	Consistencia y respuesta rápida	Patrón Outbox o cola (Kafka/RabbitMQ) a mayor escala
Testcontainers	Prueba contra el mismo motor que producción	H2 en memoria
Nginx como proxy	Mismo origen, sin CORS en producción	CORS abierto al dominio del frontend

Retos extra (sube de nivel, sin IA):

- ☐ **Paginación:** cambia `GET /api/tasks` a `Page<TaskResponse>` con `Pageable` y muestra páginas en React.
- ☐ **Relación:** agrega `Project` con `@OneToMany` a tareas, una migración `V2__...sql`, y evita el N+1 con `JOIN FETCH`.
- ☐ **Seguridad:** Spring Security con JWT; solo usuarios autenticados crean tareas y cada uno ve las suyas.

- ☐ **Documentación de API:** agrega springdoc-openapi y revisa Swagger UI en `/swagger-ui.html`.
- ☐ **Test de frontend:** Vitest + React Testing Library para el formulario.
- ☐ **Caché:** `@Cacheable` en `findById` con Redis como nuevo servicio en Compose.
- ☐ **CI:** un workflow de GitHub Actions que corra `./mvnw test` y construya las imágenes en cada push.
- ☐ **Despliegue:** sube el `docker-compose.yml` a un servidor Linux y ponlo detrás de un dominio con HTTPS.

Orden sugerido de sesiones: Pasos 1–2 (1 h) · Paso 3 (2–3 h) · Paso 4 (1.5 h) · Paso 5 (1.5 h) · Paso 6 (1 h) · Paso 7 (1 h). Sube el repo a GitHub con un README que tenga el diagrama de arquitectura y el comando `docker compose up --build`: un reclutador técnico lo puede correr en un minuto.